

Git and GitHub

Fundamentals of Computing and Data Display

Brian Kim

kimbrian@umd.edu

Motivation

Reproducibility is a key part of **how science is conducted**, and thus it is also a key part of how **data science is conducted**.

Collaboration is a key part of success in interdisciplinary fields. **Data Science** is by its nature a highly interdisciplinary field.

Git and GitHub

Organizing coding projects

- Data/ coding projects often involve many iterations
- Organizing versions particularly difficult in collaborative projects
- Manual versioning (final_paper_version_x.doc) can become cumbersome

→ Version control management systems!

- Keep track of changes (**who** changed **what** code **when**?)
- Most recent version visible, previous versions can be restored
- Work best with text files (e.g. .txt, .R, .md, .tex)

Git and GitHub

Organizing coding projects

- Data/ coding projects often involve many iterations
- Organizing versions particularly difficult in collaborative projects
- Manual versioning (final_paper_version_x.doc) can become cumbersome

→ Version control management systems!

- Keep track of changes (**who** changed **what** code **when?**)
- Most recent version visible, previous versions can be restored
- Work best with text files (e.g. .txt, .R, .md, .tex)

Git and GitHub

(Why) Git and GitHub?

- Git is a version control management system
 - <https://git-scm.com/>
 - (Most) popular software for organizing coding projects
 - Other VCSs available, e.g. Subversion (SVN)
- GitHub is a **remote host** for **local** Git repositories
 - Widely used provider for sharing and storing Git projects online
 - Can also be used as a file hoster without Git
 - GitHub alternative; GitLab

Git

Working with Git

- Command line
- Graphical User Interface (GUI)
 - Git GUI
 - <https://git-scm.com/downloads/guis>
 - <https://desktop.github.com/>
 - **RStudio**

Git configuration

- First-Time Git Setup
 - `git config --global user.name "Your name"`
 - `git config --global user.email your@email.com`
 - `git config --global core.editor editor_name`

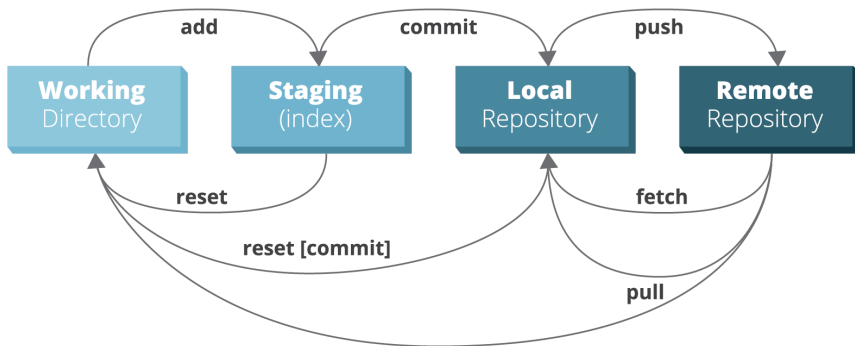
Git init

Initialize a Git repository

- Create a new (local) project
 - `git init`
- Create a new (local) R project
 - Initialize Git within an RStudio project
- Clone an existing remote project
 - `git clone`

Git workflow

Figure: Git workflow¹



¹<https://zeroturnaround.com/rebellabs/git-commands-and-best-practices-cheat-sheet/>

Example

```
> cd my_working_directory
> git init
# create some_file.R
# hack, do some work, hack
# hack
> git status # check current state of repository
> git add some_file.R
> git commit -m "Initial commit"
# hack
# more hacking
> git diff # show differences between file version
> git commit -am "Add important code"
> git log # list of previous commits
```

Git concepts

Branches

- A branch is a set of code changes that are kept separate
- Default branch created by `git init`: `main`
- Pointer to current branch: `HEAD`

`.gitignore`

- A file that tells Git which types of (e.g. temporary) files to ignore

Remotes

- A remote is an external repository to sync with
- Set up by `git clone` or `git remote add`
- Default remote name: `origin`

GitHub

Add a Git project to GitHub

- ① Create a new clean repository via the GitHub web interface
- ② Copy the resulting remote repository URL
 - `https://github.com/user_name/repo_name.git`
- ③ Add remote repo to Git project; `git remote add`
- ④ Interact with remote repo with `git push` and `git pull`

Notes!

- Public repositories are visible to everyone (open read access)
- Never push sensitive information (e.g. passwords) to a remote repository!

Example continued

```
> git remote add origin url
> git remote -v # list remote repositories
> git push origin main # push changes to GitHub
# sleep
> git pull origin main # update local repository
# hack, hack, hack, make some changes
> git commit -am "Some changes again"
> git push # push to GitHub
```

Branching

Git allows to play around with new/ experimental code via branches

- ① Create a new branch “experimental” and switch to it: `git checkout -b experimental`
 - ① `git branch experimental`
 - ② `git checkout experimental`
- ② Add and commit changes in new branch
 - ...until new branch is ready to be merged...
- ③ Switch back to main branch: `git checkout main`
- ④ Merge (no longer) experimental branch into main: `git merge experimental`
 - Resolve merge conflicts

git, GitHub and RStudio

Connect RStudio and git

- Initialize new git repository
 - ① Create a new RStudio project
 - ② Project options > Git/SVN panel > “Version control system” from “None” to “Git”
- Clone existing remote repository
 - ① Create Project > Version Control > Git
 - ② Copy&paste repo URL from GitHub
- <http://r-pkgs.had.co.nz/git.html>

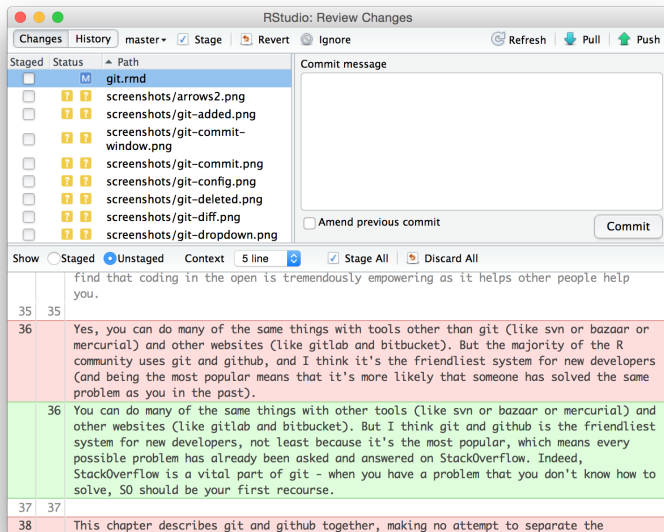
git, GitHub and RStudio

The git pane

- Diff: Opens the commit window (to review differences)
- Commit: Opens the commit window (to commit changes)
- Pull: Pull changes from remote repo
- Push: Push changes to remote repo
- Files
 -  Modified: Contents of the file have changed
 -  Untracked: A new file that Git hasn't seen before
 -  Deleted: File has been deleted
 - Tick checkbox to stage/add a file

git, GitHub and RStudio

Figure: The commit window



Resources

- Pro Git book
 - <https://git-scm.com/book/en/v2>
- Cheatsheet
 - <https://services.github.com/on-demand/downloads/github-git-cheat-sheet.pdf>
- Resource collection
 - <https://try.github.io/>
- Git and GitHub
 - <https://help.github.com/articles/git-and-github-learning-resources/>
- Git and R, RStudio
 - <http://happygitwithr.com>
 - Gandrud, C. (2015). Reproducible Research with R and R Studio. New York: Chapman and Hall/CRC.